

Bank System at Scale

Functional Requirements Document

Introduction

Bank System at Scale is a console-based banking simulation designed to model the core responsibilities of a real account-management system: account creation, deposits and withdrawals, fund transfers, and transaction history — while also being evaluated under realistic data volumes rather than a handful of toy records. The system must behave correctly under both small and large workloads, and its internal data organization must be justified by measured performance characteristics rather than assumption.

Unlike a purely correctness-focused exercise, this system is expected to be evaluated the way a production system would be: every component must protect its own invariants, expose a controlled interface rather than raw data access, and be assessed for how its behavior scales as the number of accounts and transactions grows. The system must also be able to demonstrate, through repeatable measurement, why particular internal data representations were chosen over alternatives.

The project is intentionally substantial in scope. Its purpose is to combine sound object-oriented design — encapsulation, composition, and controlled interfaces between collaborating components — with an explicit engineering discipline around performance: measuring, comparing, and justifying implementation choices under realistic scale, rather than treating performance as an afterthought.

1. Account Management

The system must support the following account-level capabilities:

- Creation of a new account with an owner name and an initial balance
- Assignment of a unique, system-generated account identifier at creation time, with no possibility of two accounts sharing an identifier
- Retrieval of an account's current balance without exposing the underlying balance field for direct modification from outside the account
- Deposits that increase an account's balance by a specified positive amount
- Withdrawals that decrease an account's balance by a specified positive amount, provided sufficient funds are available
- Rejection of any deposit or withdrawal request that is invalid (zero or negative amount, or a withdrawal exceeding the available balance), with a clear indication of failure returned to the caller

An account must be solely responsible for enforcing its own balance invariant (the balance may never become negative under normal operation). No other component in the system may directly alter an account's balance without going through the account's own controlled operations.

2. Transaction Recording

Every operation that changes an account's balance must be recorded as a transaction. Each transaction record must capture:

- A unique transaction identifier
- The type of operation (deposit, withdrawal, or transfer)
- The amount involved
- Sufficient information to reconstruct, after the fact, a complete and accurate history of an account's activity

An account must maintain its own transaction history internally, and must be able to produce a complete statement of that history on request, without requiring any external component to reconstruct it from partial information.

3. Fund Transfers

The system must support transferring funds from one account to another, subject to the following requirements:

- A transfer must be expressed as a withdrawal from the source account followed by a deposit into the destination account, each performed through that account's own public operations rather than by directly manipulating either account's internal state
- If the withdrawal from the source account fails for any reason (insufficient funds, invalid amount, invalid account), the transfer must not proceed, and the destination account must be left completely unaffected
- A successful transfer must be reflected in both accounts' transaction histories
- The component coordinating a transfer must not require privileged or direct access to either account's private data

4. Account Lookup

The system must support locating an account by its unique identifier, such that:

- A successful lookup provides access to the matching account
- An unsuccessful lookup (no matching identifier) is clearly and safely distinguishable from a successful one, without risking invalid access to a non-existent account

5. Data Volume and Scale

The system must be designed and validated to operate correctly and efficiently at scale, specifically:

- Support for at least one hundred thousand concurrently existing accounts within a single running instance of the system
- Support for processing large batches of transactions (thousands of individual deposit, withdrawal, or transfer operations) submitted together, rather than only one operation at a time
- Correct behavior must be preserved identically regardless of the number of accounts or the size of a submitted batch; scale must not introduce correctness defects

6. Performance Requirements and Benchmarking

The system must include the capability to measure and report its own performance characteristics under load. Specifically, it must be able to:

- Measure and report the time required to create a large number of accounts, under at least two different account-storage strategies, so that the cost of each strategy can be directly compared
- Measure and report the time required to apply a batch operation (such as interest accrual) across all existing accounts, under at least two different internal data organizations for that operation, so that the performance impact of data layout can be directly compared
- Measure and report the time required to process a large batch of transactions submitted together, compared against processing the same number of transactions one at a time
- Present all performance comparisons in a way that isolates the variable being tested, so that a reported difference can be attributed to a specific implementation choice rather than confounded by unrelated factors

Rationale requirement

For every case where the system offers more than one internal implementation of the same capability, the system's documentation or output must state which implementation is used by default and the measured justification for that choice.

7. Data Integrity and Invariant Enforcement

Across all scales and all operations, the system must guarantee:

- No account balance may ever become negative as a result of a normal operation
- No transaction may be lost, duplicated, or misattributed to the wrong account, including under batch processing
- No partially completed transfer may leave the system in an inconsistent state (funds appearing in neither account, or in both)
- Every account identifier issued by the system is unique for the lifetime of the running instance

8. Reporting

The system must be able to produce, on request:

- A statement of all transactions for a single specified account
- A summary of all accounts currently held by the system
- A performance report summarizing the results of any benchmarking operations that were run

9. Program Lifecycle

The system must:

- Present a means for an operator to invoke each of the capabilities described above (through a menu, command interface, or equivalent)
- Continue accepting operations until the operator explicitly chooses to exit
- Ensure that ending the program does not leave any operation partially applied or any data structure in an inconsistent state